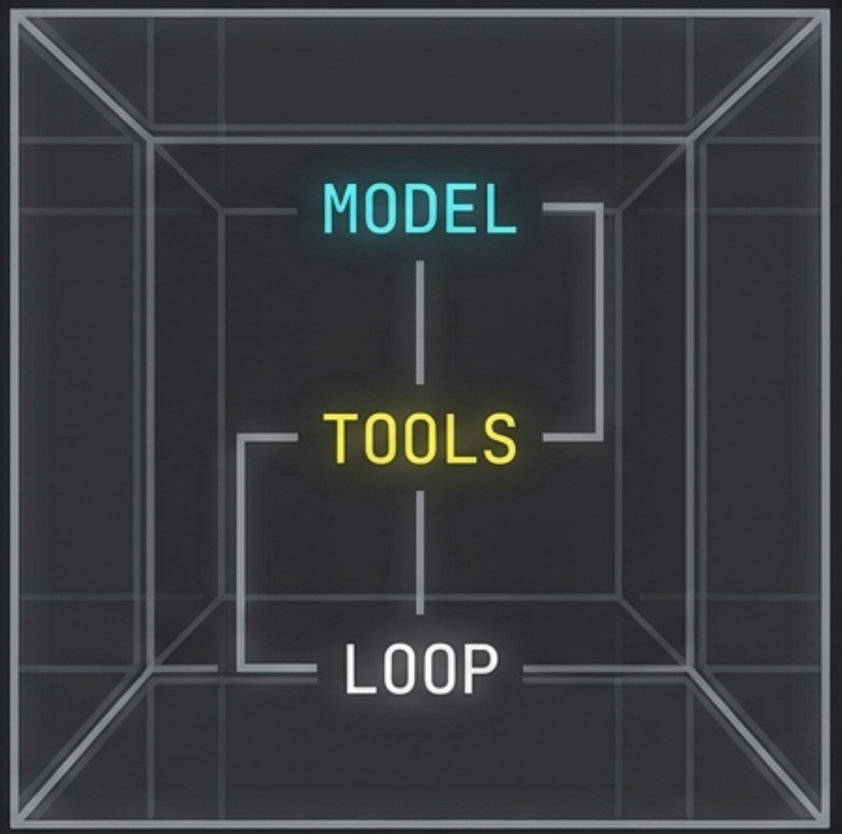


The Anatomy of a Coding Agent

A systems-based teardown of 2,938 lines of plain Node.

TARGET: Demystification | FRAMEWORK: None | DE Gemini Notebook

Myth

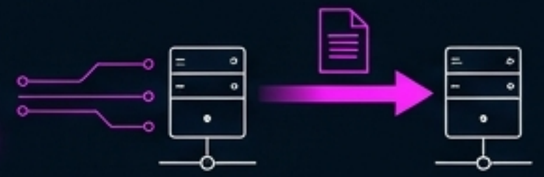
~~Autonomous Black Box~~

~~Neural Reasoning Engine~~

~~Infinite Agentic Loop~~

Three primitives and a loop

- 1 HTTP request



- 8 discrete tools



- 1 plain text array (`messages`)



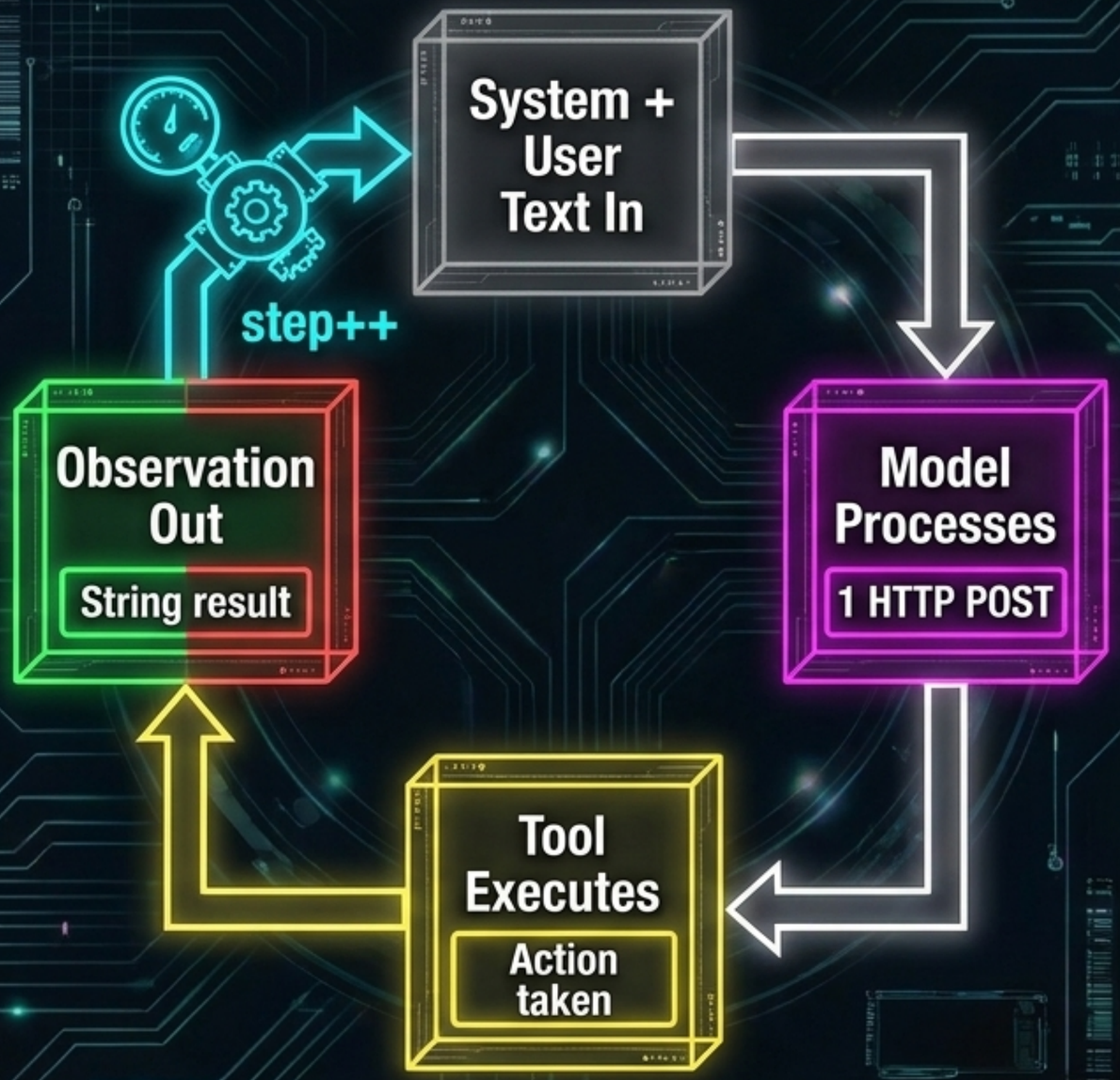
- 1 restricted folder (`workspace/`)



Strip the branding off a coding agent, and you are left with a text generator wrapped in a standard `while` loop.

Gemini Notebook

The Engine Diagram



The 6-Line Engine Code

```
messages = [system prompt, user task]

repeat up to maxSteps:
    reply = model(messages, tools)
    if reply has no tool calls:
      return reply.content
    for each tool call:
      messages.append(tool result)
```

Gemini Notebook

role: "tool"
(The Harness's Answer)

role: "assistant"
(The Model's Think + Intention)

role: "user"
(Task)

role: "system"
(Rules)



- The tool_call_id forms a mınndatory locking joint.

  Every tool call MUST receive a matching tool message, or the provider rejects the next request.

  This strict payload rule is the only way the model learns what happened in the world.

Gemini Notebook

Observation (Read)		Mutation (Write)	
`list_files`	Recursive up to depth 6. Skips .git, node_modules.	`create_file`	Fails if file exists. Forces editing over clobbering.
`read_file`	Line-numbered, paged via offset/limit.	`write_file`	Complete overwrite.
		`edit_file`	Exact-snippet replace.
Execution (Run)		**Persistence (State)**	
`run_command`	Shell in workspace root. Returns exit code + stdout/stderr.	`remember`	Appends deduped, capped note to memory.md.
		`recall`	Substring search over memory.md.

Everything returns a string. The model never touches the filesystem; it emits JSON, the harness executes it, and returns text.

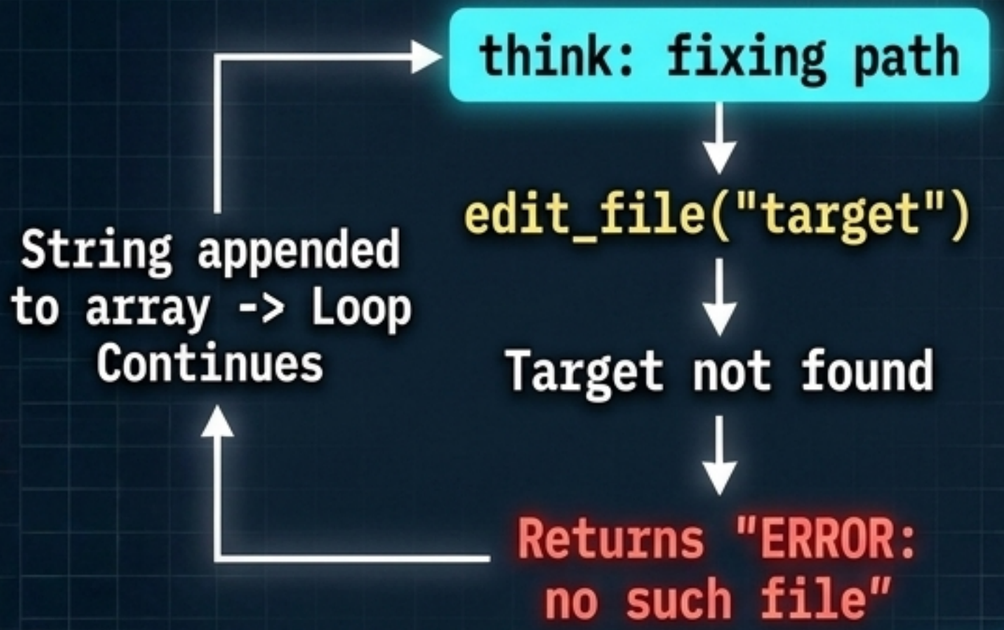
Gemini Notebook

Error as Observation, Not a Crash: Deep Dive into `edit_file`

Traditional Software



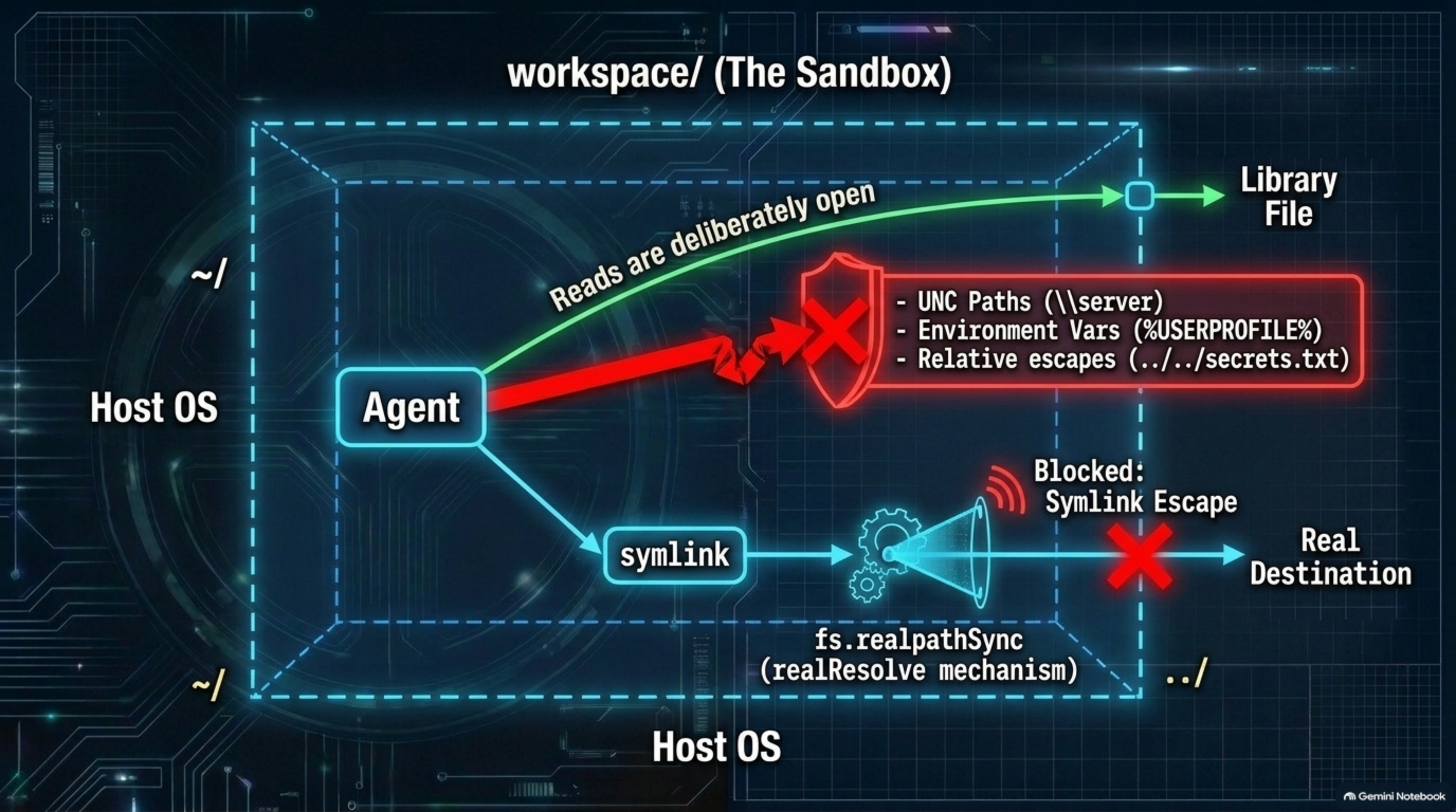
Agentic Software



Strict Rules of `edit_file`:

1. Zero matches = ERROR (Forces exact whitespace copying).
2. >1 match = ERROR (Prevents silent corruption; forces `replace_all: true`).

Gemini Notebook

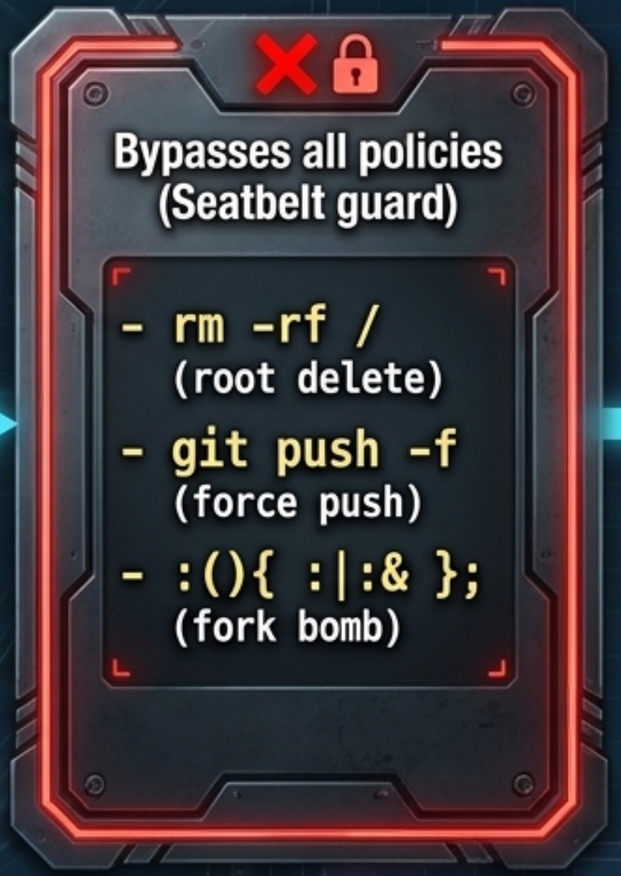

workspace/ (The Sandbox)
Reads are deliberately open
Library File
~/
Host OS
Agent
- UNC Paths (\\server)
- Environment Vars (%USERPROFILE%)
- Relative escapes (../../secrets.txt)
Blocked:
Symlink Escape
symlink
Real Destination
fs.realpathSync
(realResolve mechanism)
~/
../
Host OS
Gemini Notebook

The Command Policy (`run_command`)

Step 1: The Shell Scanner



Step 2: The Hard-Deny List



Step 3: CMD_POLICY Toggle



ask

Pauses run, waits for terminal y/N. (Default)

allow

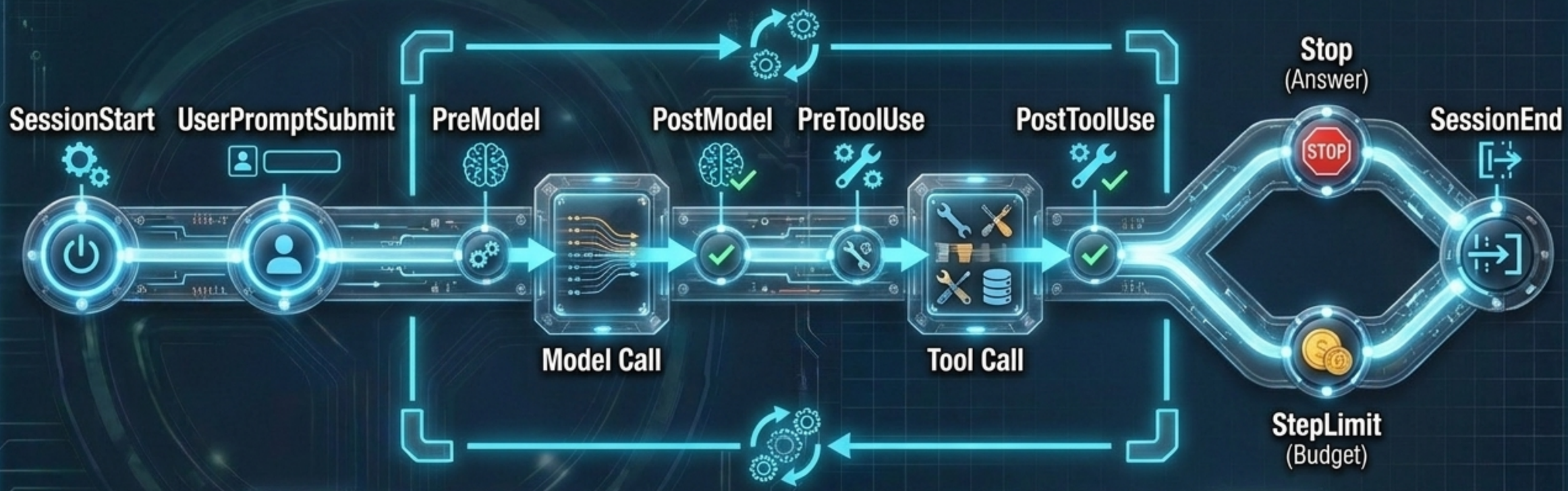
Scripted/CI mode.

deny

Shell disabled entirely.

Gemini Notebook

The Lifecycle Timeline (9 Extension Hooks)



Core Concept

Extensibility happens at named moments. A hook is just a function attached to one of these nodes. Your code never edits the core loop.

Gemini Notebook

Deep Dive: Bounding Behavior via `PreToolUse`

```
tool → run_command {"command":"git push origin main"}
```



[HOOK:PreToolUse] evaluates arguments.

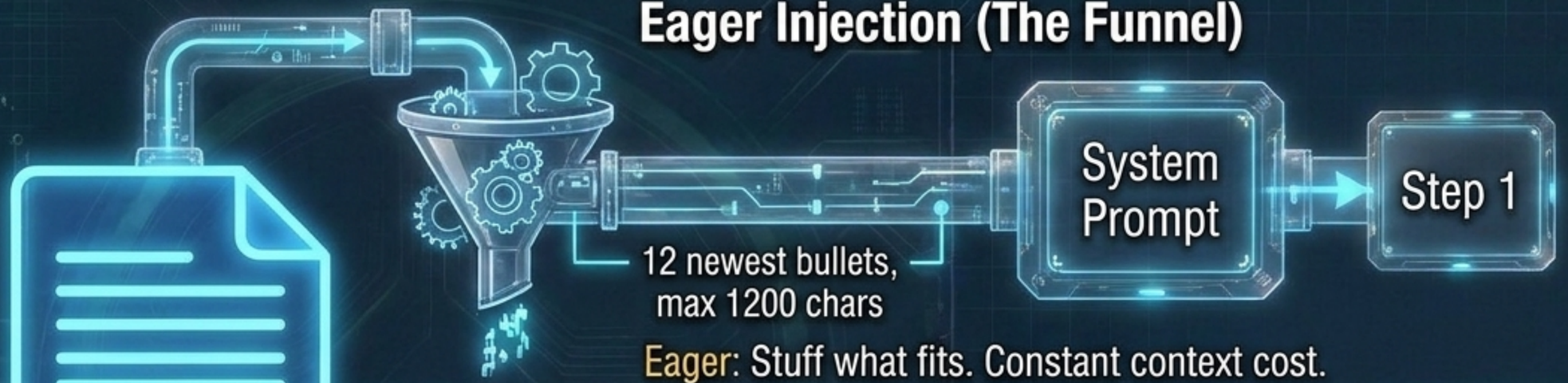
```
result ← DENIED: blocked by policy (no pushes
                 from sandbox). Try another approach.
```

A hook that blocks a tool does NOT throw an exception. It returns a DENIED string. The loop continues, satisfying the JSON payload rule, and the model reads the denial and tries a different approach on the next step.

Gemini Notebook

The Persistence Layer: `src/memory.md`

Eager Injection (The Funnel)



Eager: Stuff what fits. Constant context cost.

Lazy Retrieval (The Search)



Lazy: Retrieve when needed. Costs one tool call.

Archive of memory.md

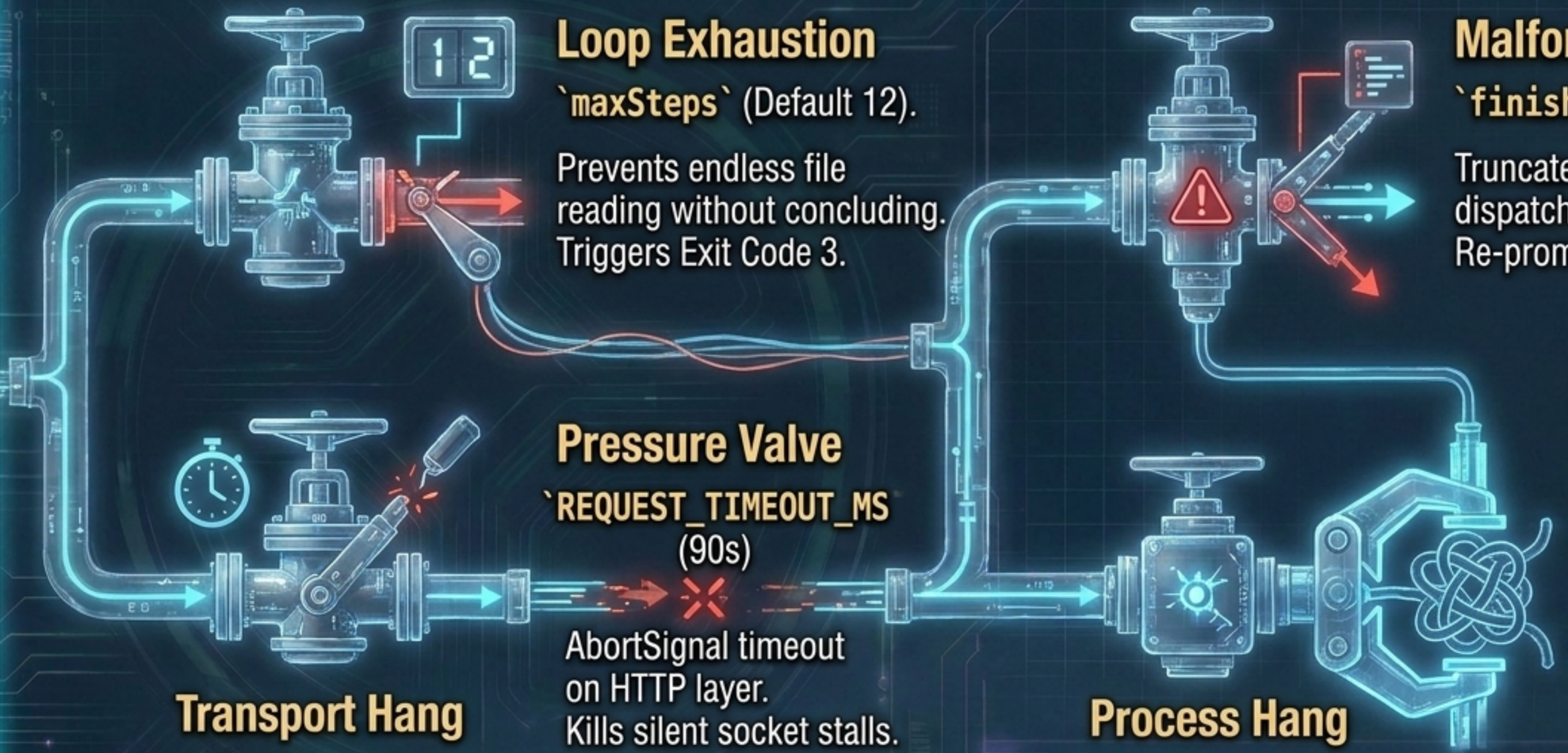
The file diffs like code.

No embeddings, no drift, no cache.

You can `cat` the agent's memory.

Gemini Notebook

Bounded Autonomy (Guarding the Loop)



Loop Exhaustion

`maxSteps` (Default 12).

Prevents endless file reading without concluding. Triggers Exit Code 3.

Malformed Intention

`finish_reason: length`

Truncated JSON is never dispatched as a half-tool call. Re-prompts to continue smaller.

Pressure Valve

`REQUEST_TIMEOUT_MS` (90s)

AbortSignal timeout on HTTP layer. Kills silent socket stalls.

Process Hang

`run_command timeout` (30s).

Kills rogue shell scripts that trap the agent.

Transport Hang

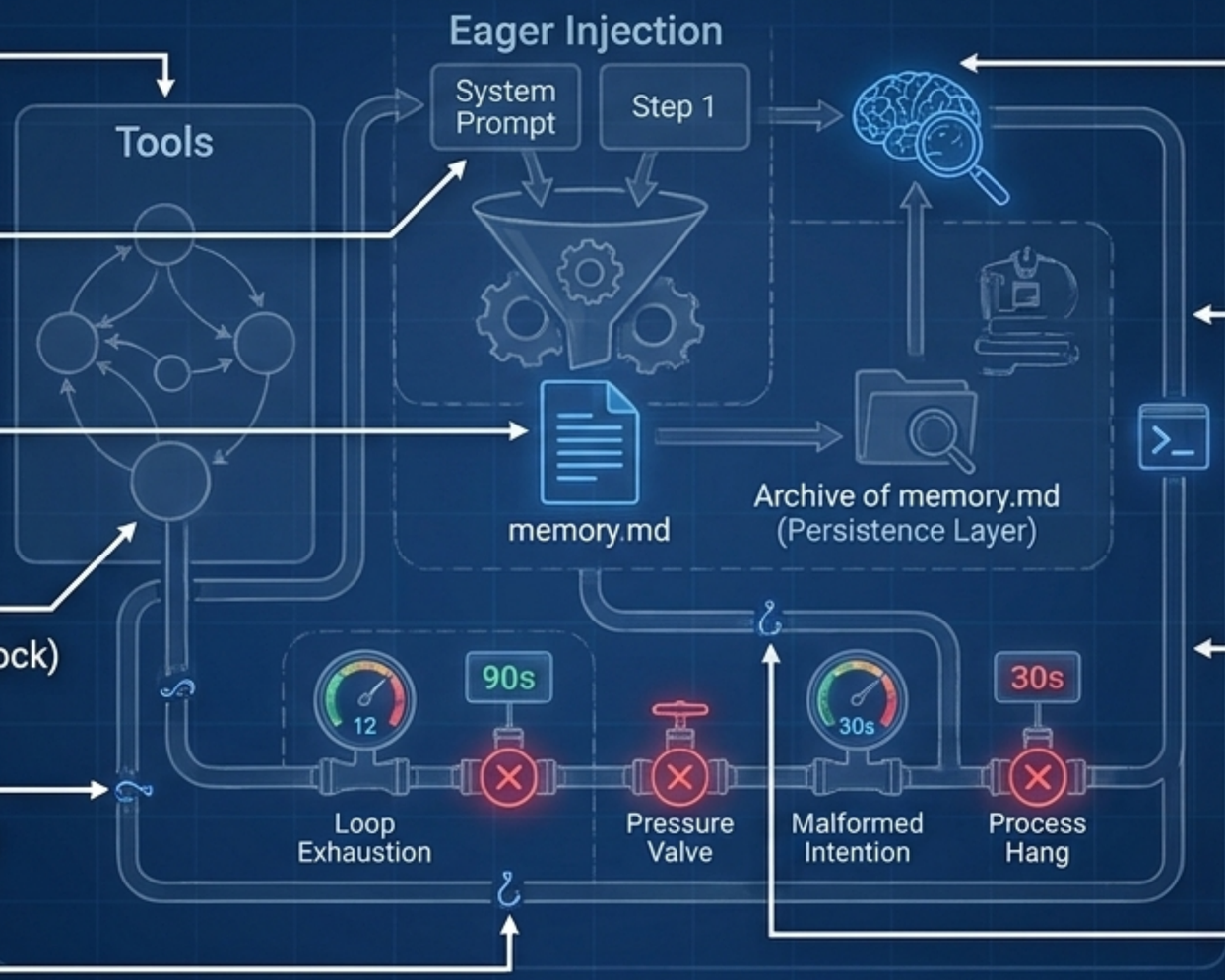
Process Hang

A loop that can run is a loop that can run forever.
Bounded autonomy replaces silent stalls with visible exit codes.

Gemini Notebook

The Blueprint of an Agent

1. Tools are defined by what they return
   `Tool.prototype.renderResult`
2. The prompt is a function of the current state
   `src/prompt.ts:getPrompt`
3. Memory is an editable Markdown file
   `src/memory.md`
4. Errors are observations, not crashes
   `src/agent.ts:runStep` (catch block)
5. Autonomy is bounded by a max step count
   `src/agent.ts:run` (`maxSteps`)
9. Extension points belong at named moments
   `src/hooks.ts`



6. The LLM is a stochastic function call generator
   `src/llm.ts:complete`
7. The loop is a state machine driven by LLM output
   `src/agent.ts:run`
8. Everything is logged to the console
   `src/logger.ts`
10. The goal is the only hardcoded state
    `src/agent.ts:run` (`goal`)

Everything else you have seen a coding agent do is built from these pieces. Now you know how the trick is done.

Gemini Notebook